Reinforcement Learning Is All You Need

Next-Token Prediction as a Special Case of Reward Maximization,
and a Research Program for RL-Native Language Models

Lemmata

Abstract

Large language models are built in two stages that are usually described as two different kinds of learning: next-token prediction (NTP) on a corpus, then reinforcement learning (RL) on rewards. This paper argues that the first stage is already the second. If the agent's action is a distribution over the next token and the reward is a strictly proper scoring rule, the expected regret of any policy equals its prediction error, exactly, at every step and every horizon; with the logarithmic score the policy gradient *is* the cross-entropy gradient. NTP is therefore reinforcement learning with the variance taken out. Through Solomonoff's theorem the RL framework then inherits the strongest known guarantee of universal prediction: one policy whose cumulative regret is finite on every task generated by any computable source. We follow the argument down to the action format language models actually use, sampled tokens, where single-sample rewards provably fail, a reward computed from two independent samples restores universality, and the natural biased group reward sharpens the distribution and deletes rare tokens. The algebraic core is machine-checked in Lean 4. We then train Transformers from scalar rewards alone, on synthetic sources with known truth, on English text and on the Lean mathematical library, and measure the price of doing so: per-step compute is the same as NTP, but sampled-token RL needs 3.6–7.1× more steps, a gap our theory attributes to the choice of scoring rule rather than to sampling. Finally we use the Lean kernel as a reward that NTP cannot use: a 30M-parameter model that never saw a proof example learns to prove 86 of 212 miniF2F test problems with 3.7 verifier calls per problem, close to a ten-tactic automation baseline at a third of its cost. We close with what the evidence does and does not support, and with a research program for training language models end to end as reward maximizers, in which pretraining is the densest reward rather than a different paradigm.

Keywords: reinforcement learning, next-token prediction, proper scoring rules, Solomonoff induction, language models, formal theorem proving, Lean

Contents

1 Introduction **2**

2 Two Paradigms and the Bridge Between Them **3**

3 Next-Token Prediction Is Reinforcement Learning **4**

4 When Actions Are Tokens **5**

5 A Machine-Checked Core **6**

6 Experiment I: Training a Transformer from Rewards Alone **7**

7 Experiment II: Rewards That NTP Cannot Use **9**

8 Toward RL-Native Language Models: A Research Program **11**

9 Where the Slogan Stops **13**

10 Conclusion **13**

A Proofs **14**

B Experimental Details **14**

1 Introduction

Every modern language model is trained twice. First it reads: billions of tokens of text, each one an exercise in guessing what comes next, scored by cross-entropy [1, 2]. Then it acts: it writes answers, programs or proofs, and is rewarded for the good ones [3, 4, 5]. The two stages are taught, implemented and discussed as two different things, supervised learning and reinforcement learning, with different losses, different codebases and different intuitions. The second is often said to be the more general: reward maximization has been proposed as a principle sufficient for intelligence itself [6].

This paper takes that suggestion literally and asks how far it can be pushed with mathematics, machine-checked proofs and experiments that a single workstation can run. The title borrows the cadence of *Attention Is All You Need* [7]. It should be read in the precise sense stated in the box below, which the rest of the paper defends, and with the limits stated in Section 9, which the paper equally defends.

Thesis

T1. NTP is a special case of RL. With the next-token distribution as action and a strictly proper scoring rule as reward, regret equals prediction error exactly; with the logarithmic score the RL gradient is the NTP gradient. NTP is RL with the variance taken out.

T2. RL therefore inherits universal prediction. One policy has finite cumulative regret on every prediction task generated by a computable source (Solomonoff, Hutter).

T3. The reduction survives in the token format language models use, provided rewards are computed from at least two samples; with one sample it provably fails.

T4. The same framework contains the rewards that NTP cannot use, such as a proof checker, so that an entire language model can in principle be specified by a single reward-maximization objective.

T1–T3 are theorems. T4 is a design claim, for which we provide a working prototype and an honest account of its limits.

Why this matters. If T1 holds, the line between "pretraining" and "RL fine-tuning" is a line between two reward signals, not between two paradigms: a dense, low-variance reward computed from data, and sparse rewards computed by verifiers, people or environments. The question "should we train language models with RL instead of NTP?" then dissolves into a more productive one: *which rewards, estimated how, and mixed in what proportion?* Several known pathologies of RL fine-tuning, most notably the collapse of output diversity, become predictable consequences of choosing a reward that is not proper (Section 4), and a principled remedy follows: keep a proper data reward in the objective. We call a model trained under one such objective an *RL-native language model* and sketch a research program for building one (Section 8).

What is new and what is not. The core universality statement (T1–T2) is a short corollary of classical work by Solomonoff [8, 9] and Hutter [10, 11, 12], and we claim no priority for it; its interest lies in how rarely it is stated in the language-model literature. The token-level analysis (T3), the machine-checked development, the experiments and the program are, to our knowledge, new in this form. The paper is written as a perspective that can be read by non-specialists: each formal section opens with its idea in words, and proofs are short or deferred to the appendix.

Roadmap. Section 2 recalls the two paradigms and the tools that connect them. Section 3 proves T1–T2 and Section 4 proves T3. Section 5 lists the machine-checked theorems. Sections 6 and 7 report experiments: training Transformers from scalar rewards alone, and learning to prove theorems from a verifier. Section 8 lays out the RL-native program, Section 9 states where the slogan stops, and Section 10 concludes.

2 Two Paradigms and the Bridge Between Them

Notation. $\mathcal{X}$ is a finite token alphabet with $m = |\mathcal{X}| \geq 2$ and $\Delta(\mathcal{X})$ the probability simplex in $\mathbb{R}^{\mathcal{X}}$; e_x is the one-hot vector of x. A *source* μ assigns to every history $x_{<t} = x_1 \cdots x_{t-1}$ a next-token distribution $\mu_t = \mu(\cdot \mid x_{<t})$; it induces a measure on infinite sequences and $\mathbb{E}_\mu$ denotes expectation under it. $\mathcal{M}_{\text{comp}}$ is the set of computable sources.

Next-token prediction. A predictor q maps histories to distributions $q_t \in \Delta(\mathcal{X})$ and is trained to minimize $\mathbb{E}_\mu \sum_t -\ln q_t(X_t)$, the cross-entropy, whose excess over the best possible value is the cumulative divergence $\sum_t \mathbb{E}_\mu \mathrm{KL}(\mu_t \| q_t)$ [13].

Reinforcement learning. An agent observes a history, takes an action, and receives a reward; a policy maps histories to (distributions over) actions and is judged by its expected total reward, or equivalently by its regret against a benchmark policy [14]. Policy-gradient methods estimate the gradient of expected reward from sampled actions and scalar rewards alone [15].

The bridge: proper scoring rules. A scoring rule $S(q, x)$ rewards a forecast $q \in \Delta(\mathcal{X})$ after outcome x is revealed. Its expected value under p is $S(q; p) = \sum_x p(x) S(q, x)$. It is *proper* if $S(p; p) \geq S(q; p)$ for all p, q, and *strictly proper* if equality forces $q = p$ [16, 17]; in words, honesty is the unique best strategy. The gap $d_S(p, q) = S(p; p) - S(q; p) \geq 0$ is the rule's divergence. Two rules matter here:

$$\text{logarithmic:} \quad S_L(q, x) = \ln q(x), \qquad d_{S_L}(p, q) = \mathrm{KL}(p \,\|\, q), \tag{1}$$

$$\text{Brier [18]:} \quad S_B(q, x) = 1 - \tfrac{1}{2}\|q - e_x\|_2^2 \in [0, 1], \qquad d_{S_B}(p, q) = \tfrac{1}{2}\|q - p\|_2^2. \tag{2}$$

The Brier identity follows from $\mathbb{E}_{X \sim p}\|q - e_X\|^2 = \|q - p\|^2 + \mathbb{E}_{X \sim p}\|p - e_X\|^2$, the cross term vanishing because $\mathbb{E}_{X \sim p} e_X = p$.

Universal prediction. Let $\mathcal{M}$ be a countable class of sources with prior weights $w_\nu > 0$, $\sum_\nu w_\nu \leq 1$. The Bayes mixture $\xi = \sum_\nu w_\nu \nu / \sum_\nu w_\nu$ dominates every member: $\xi(x_{1:n}) \geq w_\mu \mu(x_{1:n})$.

Theorem 1 (Solomonoff [9], Hutter [10])**.** *If $\xi(x_{1:n}) \geq c\, \mu(x_{1:n})$ for all $n, x_{1:n}$ and some $c > 0$, then for every n*

$$\sum_{t=1}^{n} \mathbb{E}_\mu \|\xi_t - \mu_t\|_2^2 \;\leq\; \sum_{t=1}^{n} \mathbb{E}_\mu \mathrm{KL}(\mu_t \| \xi_t) \;\leq\; \ln \frac{1}{c},$$

and $\xi_t - \mu_t \to 0$ μ-almost surely.

Taking for $\mathcal{M}$ all lower semicomputable semimeasures with weights $2^{-K(\nu)}$ (K the prefix Kolmogorov complexity) gives Solomonoff's universal predictor M; its normalization M^{norm} is a proper source that still dominates every computable μ with $c = 2^{-K(\mu)-O(1)}$ [19, 20], so

$$\sum_{t=1}^{\infty} \mathbb{E}_\mu \|M_t^{\text{norm}} - \mu_t\|_2^2 \;\le\; (\ln 2)\, K(\mu) + O(1) \qquad \text{for every } \mu \in \mathcal{M}_{\text{comp}}. \tag{3}$$

One predictor works for every computable source; the price of a source is its description length. The predictor is incomputable. A proof of Theorem 1 is recalled in Appendix A.

3 Next-Token Prediction Is Reinforcement Learning

The idea in words

Turn "guess the next word" into a game. On each turn the player does not name a word; it hands in a betting slip listing a probability for every word. The game then reveals the real next word and pays according to a proper scoring rule. Because the rule is proper, the only way to maximize expected pay is to bet one's true beliefs, and the pay lost relative to a perfectly informed player is exactly the distance between the slip and the truth. Because the player's bets never change what text comes next, there is nothing to explore and nothing to plan: playing well today never costs anything tomorrow. Playing this game well and predicting well are the same skill.

Definition 1 (Prediction environment)**.** Fix a source μ and a scoring rule S. In the environment E_μ^S, at step t the agent observes $x_{<t}$, outputs an action $q_t \in \Delta(\mathcal{X})$, the environment draws $X_t \sim \mu_t$ independently of q_t, reveals it, and pays $S(q_t, X_t)$. The regret of a policy π is $R_T(\pi, \mu) = \mathbb{E}_\mu \sum_{t \le T} [S(\mu_t, X_t) - S(q_t, X_t)]$, measured against the informed policy that outputs μ_t.

Lemma 1 (Regret equals prediction error)**.** *For every policy, source and horizon, $R_T(\pi, \mu) = \sum_{t \le T} \mathbb{E}_\mu\, d_S(\mu_t, q_t)$. For the Brier reward $R_T = \frac{1}{2} \sum_t \mathbb{E}_\mu \|q_t - \mu_t\|^2$; for the log reward $R_T = \sum_t \mathbb{E}_\mu \mathrm{KL}(\mu_t \| q_t)$. If S is proper the informed policy is optimal among all policies, at every horizon.*

Proof. Condition on $x_{<t}$: then q_t is fixed and $X_t \sim \mu_t$, so the conditional expected regret of step t is $d_S(\mu_t, q_t)$. Sum and take expectations. Actions do not influence observations, so step-wise optimality is global optimality. □

Corollary 1 (NTP is RL with the variance taken out)**.** *Let $q_t = \mathrm{softmax}(z_\theta(x_{<t}))$. In $E_\mu^{S_L}$ the expected reward of step t given the observed token is $\ln q_t(x_t)$, which is minus the NTP loss. Its gradient is the cross-entropy gradient; no sampling of actions is involved because the action is the whole distribution and the reward is differentiable in it.*

The corollary is elementary, and that is the point. In the vocabulary of RL, ordinary pretraining is an *exact* policy-gradient method in a non-reactive environment: the reward of every possible action is known in closed form once the next token is revealed, so the expectation over actions can be taken analytically. Sampling-based RL estimates the same quantity with noise. Section 6 measures how large that noise is in practice.

Theorem 2 (Universality in prediction environments)**.** *Let π^ξ output the Bayes mixture prediction ξ_t. For every $\mu \in \mathcal{M}$, with the Brier reward $\sup_T R_T(\pi^\xi, \mu) \le \frac{1}{2} \ln(1/w_\mu)$, and with the log reward $R_T(\pi^\xi, \mu) \le \ln(1/w_\mu)$ for all T. The policy that outputs M_t^{norm} satisfies, under the Brier reward,*

$$\exists \pi \;\; \forall \mu \in \mathcal{M}_{\text{comp}} \;\; \forall T: \qquad R_T(\pi, \mu) \le \tfrac{1}{2}(\ln 2) K(\mu) + O(1).$$

Proof. Lemma 1 and Theorem 1 with (3). □

Remark 1 (Reading the quantifiers)**.** One policy, chosen before any data is seen, works for every computable source; the bound depends on the source but not on the horizon. Bounded cumulative regret means that the average regret vanishes and predictions converge, not that the policy is right from the start. The policy is incomputable: the theorem is an existence result about what the RL framework can express and achieve, not about what a particular algorithm will find. Hutter already analyzed the universal mixture as a decision maker under general loss in non-reactive environments [11] and treated sequence prediction as a special case of AIXI [12]; Theorem 2 adds only the exact regret identity that makes the transfer loss-free.

4 When Actions Are Tokens

The idea in words

A language model does not hand in a betting slip; it *samples* a token. If the reward looks at one sampled token only, then the expected reward is an average of the rewards of individual tokens, and the best strategy is always to put all probability on the single best token. No reward design, however clever, can make "tell the truth about the distribution" optimal. Two samples change this: comparing them reveals how concentrated the policy is, and a reward that subtracts their agreement turns honesty back into the optimum.

Definition 2 (K-sample token environment)**.** At step t the policy specifies $\pi_t \in \Delta(\mathcal{X})$, the agent draws $A_1, \dots, A_K \overset{\text{iid}}{\sim} \pi_t$, the environment draws $X_t \sim \mu_t$, and pays $\rho(A_{1:K}, X_t)$. Write $U_\rho(q;p) = \mathbb{E}_{A\sim q^{\otimes K}, X\sim p}\,\rho(A,X)$. Lemma 1 holds verbatim with $d(p,q) = U_\rho(p;p) - U_\rho(q;p)$.

Proposition 1 (One sample is never enough)**.** *For $K = 1$, $U_\rho(q;p) = \sum_a q(a)\sum_x p(x)\rho(a,x)$ is linear in q. For every ρ and every p with two or more support points, some point mass does at least as well as p; p is never the unique optimum.*

What if a regularizer of the policy is added, as practitioners do with entropy bonuses and KL penalties? The answer is sharp.

Theorem 3 (Only quadratic objectives are reachable)**.** *Let $R_{ax} = \rho(a,x)$ and let $\Omega : \Delta(\mathcal{X}) \to \mathbb{R}$ be continuous, and differentiable on the relative interior along the tangent space $T_0 = \{v : \sum_a v(a) = 0\}$. If every interior p maximizes $J_p(q) = U_\rho(q;p) + \Omega(q)$, then $J_p(p) - J_p(q) = \frac{1}{2}(q-p)^\top R(q-p)$ for all interior p and all q. Hence the induced divergence is a fixed quadratic form, Ω is quadratic up to an affine term, and no entropy or KL regularizer, and no single-sample reward with any smooth regularizer, yields the logarithmic score.*

Proposition 2 (Two samples suffice)**.** *For $K = 2$ and $\rho_2(a,a',x) = \mathbf{1}\{a = x\} + \mathbf{1}\{a' = x\} - \mathbf{1}\{a = a'\}$,*

$$U_{\rho_2}(q;p) = 2\langle q,p\rangle - \|q\|^2 = \|p\|^2 - \|q-p\|^2.$$

The reward is strictly proper on the whole simplex and its regret is exactly $\|q-p\|^2$.

Proof. $\mathbb{E}\mathbf{1}\{A = X\} = \mathbb{E}\mathbf{1}\{A' = X\} = \langle q,p\rangle$ and $\mathbb{E}\mathbf{1}\{A = A'\} = \|q\|^2$ by independence. □

The reward ρ_2 is the fair Brier score of Ferro [21] for a two-member ensemble. Its policy gradient uses only sampled tokens: $\nabla U_{\rho_2} = \mathbb{E}[\rho_2(A,A',X)(\nabla \ln \pi(A) + \nabla \ln \pi(A'))]$.

Theorem 4 (Universality for token-sampling policies)**.** *In the two-sample token environment with reward ρ_2, the policy that samples from ξ_t has $\sup_T R_T \le \ln(1/w_\mu)$ for every $\mu \in \mathcal{M}$, and the policy that samples from M_t^{norm} has $\sup_T R_T \le (\ln 2)K(\mu) + O(1)$ for every computable μ.*

Groups: fair versus biased. Group-based methods such as GRPO [4] draw K samples per context. Estimating $\|q\|^2$ by collisions among all K^2 ordered pairs, diagonal included, is the natural "plug in the empirical distribution" choice. It is biased:

Proposition 3 (Biased group rewards sharpen and truncate)**.** *With $c = (K-1)/K$, the biased (V-statistic) group reward has $U(q;p) = 2\langle q,p\rangle - c\|q\|^2 - 1/K$, whose unique maximizer is the Euclidean projection of p/c onto the simplex: $q^\star(a) = (\frac{K}{K-1}p(a) - \tau)_+$. It increases every likelihood ratio between surviving tokens and assigns zero probability to every token with $p(a) \le c\tau$.*

For $K = 2$ and $p = (0.5, 0.3, 0.15, 0.05)$ the biased optimum is $(0.7, 0.3, 0, 0)$: half of the vocabulary disappears. The fair (U-statistic) version, which leaves out the diagonal, has $U = \|p\|^2 - \|q-p\|^2$ for every $K \ge 2$.

Proposition 4 (No finite number of samples reaches the log score)**.** *For any K and ρ, $U_\rho(q;p)$ is a polynomial of degree at most K in q, so its divergence is bounded on $\Delta(\mathcal{X}) \times \Delta(\mathcal{X})$ and is never a positive multiple of* $\mathrm{KL}(p\|q)$.

Proposition 4 has a practical reading that Section 6 confirms: a learner that only ever sees sampled tokens and scalar rewards is confined to Brier-type objectives, and pays whatever Brier-type objectives cost relative to the logarithmic score. Proofs of Theorem 3 and Proposition 3 are in Appendix A.

A lesson for RL fine-tuning. RL with verifiable rewards scores one sampled response, so its objective is linear in the policy and its optima are concentrated on reward-maximizing responses (Proposition 1); with a KL penalty the optimum is a reweighting $\pi^\star \propto \pi_{\mathrm{ref}} e^{r/\beta}$ of the reference [22, 23, 24]. Loss of diversity is thus the expected behavior of these objectives, not an accident of training, consistent with the empirical finding that RL-trained models can be overtaken by their base models at large pass@k [25]. The theory also names the remedy: keep a strictly proper reward on the data in the objective. Section 7 tests this.

5 A Machine-Checked Core

The finite, algebraic core of Sections 3 and 4 is formalized in Lean 4 [26] with Mathlib [27]. Distributions are non-negative functions on a finite type summing to one; sources and policies map token histories to distributions, and sources do not read actions; value and cumulative error are defined by recursion on the horizon and integrate each next token against the source, so they include the expectation over random histories. All theorems below build with Lean 4.34.1 and Mathlib v4.34.1 without `sorry`, and depend only on the standard axioms `propext`, `Classical.choice` and `Quot.sound`.

Lean theorem	Content
`brier_gap`	$S_B(p;p) - S_B(q;p) = \frac{1}{2}\|q-p\|^2$
`reward_bounds`, `truthful_unique`	the Brier reward lies in $[0,1]$ and is strictly proper
`regret_eq_half_error`	Lemma 1 for the Brier reward, every horizon and history
`universal_transfer`	a horizon-free error bound B for one policy on a class gives regret $\le B/2$
`single_sample_not_strict`	Proposition 1
`two_sample_gap`, `two_sample_unique`	Proposition 2: regret identity and strict propriety
`regretU_eq_div`	Lemma 1 for an arbitrary single-step expected reward
`token_regret_eq_error`, `token_universal_transfer`	Theorem 4 in conditional form
`biased_two_sharpens`	Proposition 3 for $K = 2$, unclipped case

Table 1: Synthetic source with known truth (order-2 Markov, 16 tokens, 4000 steps). Mode accuracy is the fraction of contexts in which the model's most likely token is the true most likely token.

Method	Learner sees	$\|q-\mu\|^2$	$\mathrm{KL}(\mu\|q)$	Mode acc.	Theory
ntp	true token	0.0006	0.005	0.95	baseline
brier	scalar reward	0.0010	0.027	0.95	proper (Lemma 1)
pg8, fair	scalar reward	0.0014	0.033	0.95	proper (Prop. 2)
pg2, fair	scalar reward	0.0041	0.074	0.92	proper, noisier
pgv2, biased	scalar reward	0.0574	0.532	0.93	sharpens (Prop. 3)
pg1	scalar reward	0.903	9.74	0.19	collapses (Prop. 1)
pg1 + entropy 0.05	scalar reward	0.219	0.813	0.96	mode only (Thm. 3)

The Solomonoff bound enters `universal_transfer` and `token_universal_transfer` as an explicit hypothesis, not as an axiom. Theorem 1, the universal semimeasure and measures on infinite sequences are not formalized; connecting this development to a formalized Solomonoff bound [28] is future work.

6 Experiment I: Training a Transformer from Rewards Alone

The question

The theorems say that an RL agent *can* predict as well as anything. Does an ordinary Transformer, trained by ordinary policy gradient from nothing but scalar rewards, actually learn to predict? And what does it cost compared with NTP?

Training signals. The model is a standard decoder-only Transformer; only the training signal changes. In every RL mode the true next token is hidden inside the environment and used only to compute a scalar reward; no gradient flows from it. We compare ntp (cross-entropy); brier (action = the whole distribution, reward S_B, differentiable in the action); pg$K$ (action = K sampled tokens, fair group reward, REINFORCE with a batch baseline and per-sample credit whose expectation is exactly $\nabla(2\langle q,p\rangle - \|q\|^2)$); pgv$K$ (the biased group reward); and pg1 (one sampled token, reward $\mathbf{1}\{A=x\}$, optionally with an entropy bonus).

Three settings. (i) A synthetic order-2 Markov source over 16 tokens with Dirichlet(0.5) transition rows, where the true μ_t is known and the distance to it can be measured; 4 layers, width 128, 4000 steps. (ii) Character-level Lean 4 code: a 9 MB subset of Mathlib, 313 symbols, 6 layers, width 256, about 5M parameters, 4000 steps. (iii) BPE-level Lean: all of Mathlib, a byte-level BPE vocabulary of 8192, 30.6M training tokens with held-out files for validation, 8 layers, width 512, 29.7M parameters, 15,000 steps of 32×512 tokens. All runs use one laptop GPU (RTX PRO 4000 Blackwell, 16 GB) and one seed.

Result 1: rewards alone suffice to learn the distribution. Table 1 confirms every prediction of Section 4 on the synthetic source. The fair sampled-token reward recovers the true conditional distribution to within $\|q-\mu\|^2 = 0.0014$ (NTP: 0.0006); the single-sample reward collapses; the entropy bonus gets the mode right and the distribution wrong; the biased group reward sharpens. On real code the RL-trained models produce recognizable Lean (declarations, `by simp`, `rfl`, attributes), and their validation loss tracks NTP's at a distance (Fig. 1).

Result 2: the price is statistical, not computational. Per training step, RL and NTP cost the same: the forward and backward passes dominate, and sampling K tokens, counting

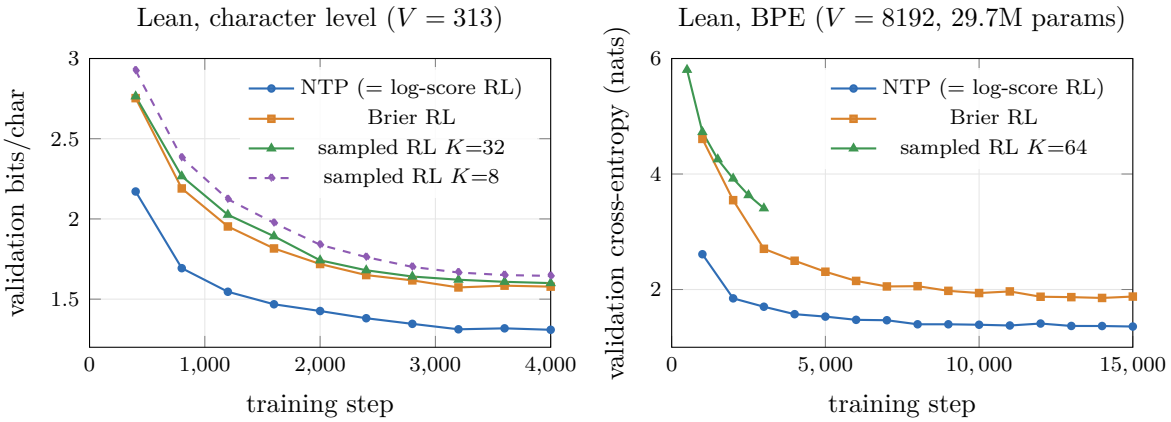


Figure 1: Validation loss of Transformers trained from rewards alone, compared with NTP. All curves descend; the RL curves lag by a roughly constant factor in steps that grows with the vocabulary.

Table 2: Per-step cost relative to NTP (median of three interleaved rounds on a shared GPU; ratios are the meaningful quantity).

Scale	Method	Time per step	Peak memory
Character level, $V = 313$	brier	0.93×	1.04×
	pg $K = 8$ / 32	0.95× / 0.94×	1.04× / 1.05×
	pg $K = 128$	1.11×	1.66×
BPE, $V = 8192$	brier	1.15×	1.12×
	pg $K = 8$ / 64	1.13× / 1.21×	1.44× / 1.44×

collisions and computing $\|q\|^2$ are cheap (Table 2). The difference lies in how much each step teaches. Table 3 reports, for each RL run, the number of steps NTP needed to reach the same quality (by interpolating the NTP curve), and the ratio.

Result 3: the gap is mostly the scoring rule, as Proposition 4 predicts. The ratios decompose. Comparing brier with NTP isolates the choice of scoring rule (Brier versus logarithmic) with no sampling at all: 3.6–3.7× at small vocabularies and 7.1× at $V = 8192$. Comparing pg$K$ with brier isolates sampling noise: an extra 1.05–1.25× for $V \leq 313$, decreasing with K, and 1.4× at $V = 8192$ with $K = 64$ (Brier reaches the cross-entropy that pg64 attains at step 3000 after about 2130 steps). The scoring-rule term grows with the vocabulary for a simple reason: the logit gradient of the log score on the correct token is $1 - q(x)$, whereas that of the Brier score is $q(x)(1 - 2q(x) + \|q\|^2) \approx q(x)$, which vanishes when $q(x)$ is small, and early in training $q(x) \approx 1/V$. At BPE scale this is decisive: at every step the NTP model scores higher than the Brier-trained model *even on the Brier score* (0.284 versus 0.273 at step 15,000). The metric also matters: on squared error, the Brier rule's own divergence, Brier-RL needs only 1.3× the NTP steps on the synthetic source. Since sampled-token learners are confined to polynomial, Brier-type objectives (Proposition 4), they inherit this price.

Table 3: Statistical efficiency: steps an RL run took, steps NTP needed to reach the same value, and their ratio.

Setting	Metric	Method	RL steps	NTP-equivalent	Ratio
Synthetic, $V = 16$	$\mathrm{KL}(\mu\|q)$	brier	4000	1077	3.7
		pg8	4000	870	4.6
		pg2	4000	549	7.3
	$\|q - \mu\|^2$	brier / pg8	4000	3196 / 2932	1.3 / 1.4
Lean char, $V = 313$	bits/char	brier	4000	1126	3.6
		pg32	4000	1052	3.8
		pg8	4000	931	4.3
Lean BPE, $V = 8192$	cross-entropy	brier	15000	2117	7.1
		pg64	3000	578	5.2

What Experiment I shows

RL from scalar rewards does learn next-token prediction in a real Transformer, at the same cost per step. The efficient instance of this RL is the one that takes the expectation over actions analytically with the logarithmic score, which is NTP itself. Any RL-native design should therefore keep this instance as its data reward and spend sampling only where no analytic expectation exists.

7 Experiment II: Rewards That NTP Cannot Use

The question

If NTP is the dense, analytic corner of RL, what does the rest of RL buy? The clearest case is a reward that has no corpus behind it: a proof checker. NTP can only imitate proofs that already exist; RL can learn from the checker alone. Can a small model that has never seen a proof of the target problems learn to produce them, and what goes wrong?

Setup. Each experiment starts from a model pretrained on Lean code (Section 6). For each statement the model samples tactic proofs; Lean checks them; the reward is 1 if Lean accepts the proof and 0 otherwise, with group-normalized advantages. Optionally the stage-1 proper reward on corpus text is kept as an *anchor* with weight λ, so that the objective is a single mixture of a data reward and a verifier reward. No proof examples are ever shown.

Toy propositions. With the character-level model, randomly generated core-Lean goals from 12 templates (introduction and elimination of $\wedge, \vee, \to$, simple arithmetic) and one-line proofs, pass@1 on 48 held-out goals rose from 1.3% to 22.9% with the verifier reward alone, and to 52.1% with the fair-Brier anchor ($\lambda = 0.5$). Without the anchor the policy collapsed onto a single answer (`simp`) within 50 steps and stayed there; with it, the model learned to cite the relevant hypothesis (`simp [h2]`), producing nine distinct correct proof forms. This is Section 4 in action: a pure 0–1 task reward is mode-seeking, and a proper data reward preserves the diversity that further learning needs.

miniF2F, one line. With the BPE model we used the Lean 4 statements of miniF2F [29] distributed with DeepSeek-Prover-V1.5 [30]; 423 of 488 elaborate under our Mathlib version. RL runs on the `valid` split and evaluation on the 212 `test` problems. The reference point is a strong automation baseline: trying each of ten tactics (`grind`, `nlinarith`, `omega`, `linarith`,

Table 4: miniF2F test (212 problems), Lean 4 and Mathlib. "Beyond" counts problems the ten-tactic baseline does not solve; "calls" is the mean number of Lean checks per problem. One-line rows: mean over two seeds, 60 RL steps, 16 samples per problem at evaluation.

Method	Solved	Beyond	Calls
Ten automation tactics, each tried once (baseline)	89	–	10
Tactic-only best-first search, no model	94	5	20.3
One-line proofs, sampled			
pretrained model, no RL	23	0	16
RL: verifier reward + anchor $\lambda = 0.5$	78	0	16
RL: + larger groups, stronger anchor, diversity reward	84	0.5	16
Multi-line proofs, best-first search with goal states			
pretrained model, policy only	20	0	14.9
RL (expert iteration), policy only	86	2	**3.7**
pretrained model + automation candidates	93	5	47.0
RL + automation candidates	94	5	21.6

`ring_nf`, `aesop`, `norm_num`, `decide`, `simp`, `positivity`) solves 89 test problems. Before RL the model solves 23 (16 samples per problem). After 60 RL steps it solves 76–80 with the verifier reward and the anchor; adding larger groups and a diversity reward lifts this to 84 on both seeds (Table 4). Across all runs RL solves 88 problems, three of them beyond the baseline through tactic combinations such as `ring <;> norm_num <;> grind`.

Reward hacking appears exactly where the reward is not proper. Our diversity bonus first rewarded textually distinct correct proofs; the policy learned to pad `grind` with irrelevant lemma hints (`grind [add_assoc]`, `grind [mul_comm]`). Rewarding distinct tactic sequences instead, it learned to repeat tactics (`grind <;> grind <;> grind`). Both are instances of reward hacking [31]: a diversity measure defined on the surface form can be satisfied by changing the surface form. A strictly proper data reward is immune to this particular failure, because its expectation is maximized only by the true distribution, whatever the surface; this is a practical argument for proper scores as the diversity-preserving term.

miniF2F, multi-line, with goal states. Finally we let the model write proofs step by step. After each step Lean reports the remaining goals, which are shown to the model as a comment; best-first search orders partial proofs by cumulative log-probability and discards steps that do not change the goal. RL is expert iteration [32, 33]: search on the `valid` split, reward every step of every accepted proof, raise its probability under the anchor, repeat twice. Table 4 and Fig. 2 report the results. The search with automation tactics as additional candidates solves 94 problems, above the 89 baseline. The honest reading, however, is that the credit belongs to the search: a search over the same ten tactics with no model at all also solves 94, at similar cost, and all five problems beyond the baseline are two-tactic combinations such as `aesop` followed by `nlinarith`. What RL demonstrably buys is efficiency: searching with the RL-trained policy alone solves 86 problems with 3.7 Lean calls per problem on average, against 20 before RL; the ten-tactic baseline needs 10 calls for 89 and the tactic-only search 20.3 calls for 94. With automation candidates added, RL halves the verifier calls (47.0 $\rightarrow$ 21.6) and the wall-clock time (24 $\rightarrow$ 9 minutes) at the same solve rate.

Two pitfalls worth reporting. Both changed our numbers substantially and are easy to repeat. First, batching many candidate proofs into one Lean file lets a malformed candidate (for example, a lone comment after `by`) push its parse error onto the next theorem, so that it is scored as correct; error attribution must account for this, and comments and search commands

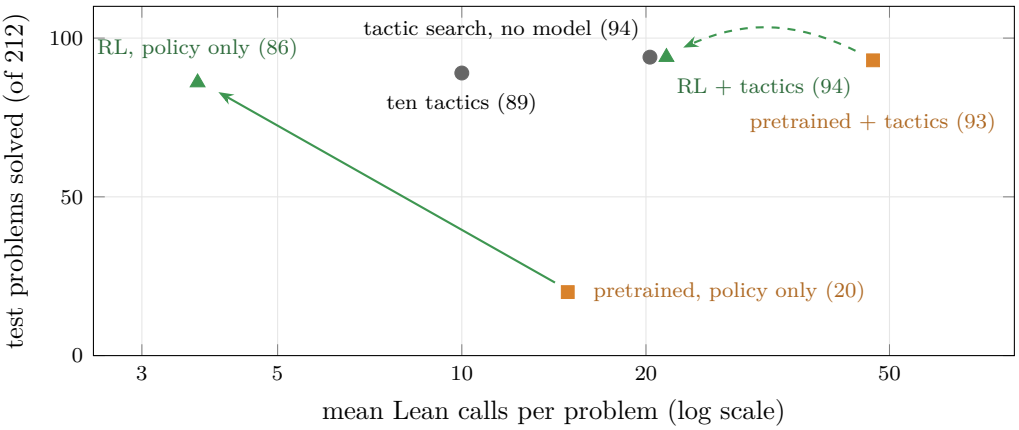


Figure 2: Solve rate against verifier cost on miniF2F test. Arrows show the effect of RL from the verifier. RL turns a model that cannot prove into one that reaches 97% of the automation baseline at a third of its cost, but at this scale it does not exceed what search over automation already achieves.

such as `exact?` must be rejected. Second, byte-level BPE merges the newline after `by` with the following indentation, so a prompt that ends in a bare newline is out of distribution: our model then produced comments and new declarations, and assigned probability zero to `grind`. Ending the prompt at `by` fixed this.

What Experiment II shows

A verifier is a reward NTP cannot use, and RL from it turns a model that has never seen a proof into a competent, cheap prover, with no demonstrations. Pure task rewards collapse the policy; a proper data reward in the same objective counteracts collapse and, unlike surface-level diversity bonuses, cannot be gamed by rewriting. At 30M parameters the learned policy reproduces, and makes far cheaper, the competence latent in its pretraining and in Lean's automation; exceeding that competence requires capacity this study did not have.

8 Toward RL-Native Language Models: A Research Program

The usual pipeline treats a language model as a predictor first and an agent second. The results above suggest treating it as an agent throughout, with prediction as the agent's densest reward. We call this the RL-native view and state it as one objective.

One objective. For a policy π_θ over token sequences, a data distribution $\mathcal{D}$, a strictly proper scoring rule S and task rewards r_j with environments $\mathcal{E}_j$,

$$J(\theta) \;=\; \lambda \underbrace{\mathbb{E}_{x\sim\mathcal{D}} \sum_t S(\pi_\theta(\cdot \mid x_{<t}), x_t)}_{\text{data reward: analytic, dense, proper}} \;+\; \sum_j \beta_j \underbrace{\mathbb{E}_{e\sim\mathcal{E}_j,\, y\sim\pi_\theta(\cdot|e)}\, r_j(e,y)}_{\text{task rewards: sampled, sparse}} . \tag{4}$$

With $S = S_L$ and $\beta_j = 0$ this is pretraining. With $\lambda = 0$ it is RL with verifiable rewards and its collapse. Everything in between is a single reward-maximization problem, with one schedule for λ and β_j in place of a hand-off between stages. Section 4 explains why the data term should stay ($\lambda > 0$): it is the only term in (4) whose optimum is a distribution rather than a mode, and Experiment II shows that it is the term that cannot be gamed by surface rewriting. Fig. 3 contrasts the two views.

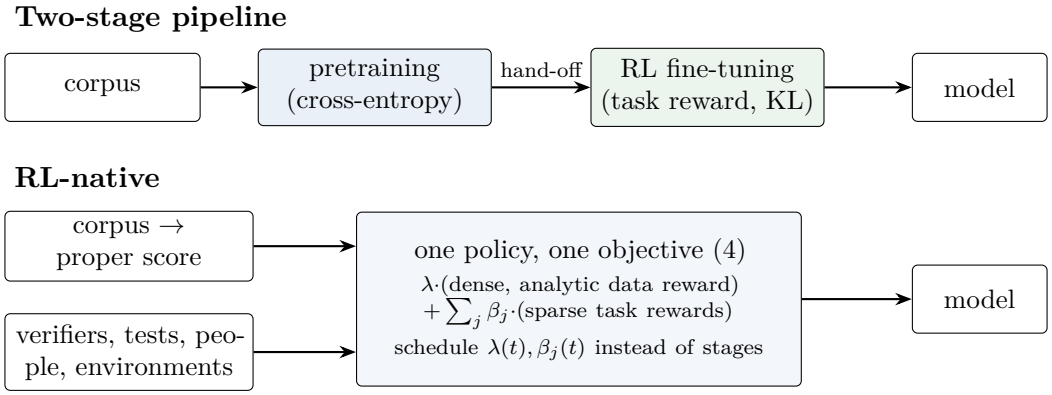


Figure 3: The two-stage pipeline and the RL-native view. In the latter, pretraining is not a separate paradigm but the densest, analytically computable reward in a single objective.

What has to be solved. The program is only as good as its open problems. We list those our analysis and experiments expose, in rough order of urgency.

P1. Keep the analytic corner. Wherever an expectation over actions can be computed in closed form, as it can for next-token rewards on data, compute it, and use the log score. Experiment I shows that giving this up costs 3.6–7.1× in steps. Sampling should be reserved for rewards that only exist as black boxes.

P2. Variance and vocabulary. Sampled-token learning degrades as the vocabulary grows, because the chance that a sample hits the observed token is small. Control variates that exploit the known structure of (4), importance sampling from a proposal concentrated near the data token, and hierarchical or factorized action spaces are natural directions, as is a theory of how many samples per context a fair group reward needs as a function of the entropy of μ_t.

P3. Proper rewards at the level of responses. Our token-level results lift directly only to rewards on whole responses when those rewards are themselves proper scores of a distribution. Designing sequence-level fair rewards that are cheap to estimate from a group of sampled responses, and whose optimum is a calibrated distribution over correct answers rather than one answer, is the response-level analogue of Proposition 2.

P4. Semantic, not syntactic, diversity. Experiment II shows that diversity defined on surface forms is gamed immediately. Diversity measured on meaning (for proofs: proof terms or goal trajectories; for programs: behavior on tests) is needed wherever a proper data reward is unavailable.

P5. Verifiable environments as the new corpus. If pretraining is the data reward, then curated environments with checkable rewards, such as formal mathematics, program synthesis and simulators, play the role that text corpora play today. Their scale, diversity and curricula become first-class research objects [33, 34].

P6. Exploration and irreversibility. All guarantees above live in non-reactive environments. In interactive settings with irreversible actions no policy matches the informed optimum (Section 9); RL-native models that act in the world need weaker, history-relative optimality notions and safe exploration.

P7. Scaling laws for the mixture. How should λ and β_j evolve with compute? Our data suggest that early training should be dominated by the analytic data reward, which is most informative per step, and that sparse rewards pay off once the prior places non-negligible probability on rewarded behavior, as the jump from 20 to 86 problems illustrates.

Table 5: Readings of "reinforcement learning is all you need" and their status.

Reading	Status
RL can express every probabilistic prediction task, NTP included	True with a strictly proper reward (Lemma 1, Corollary 1)
One policy has bounded regret on all computable prediction environments	True, incomputable policy (Theorems 2, 4)
Same, with single-sample token rewards	False (Proposition 1, Theorem 3)
Ordinary policy gradient learns NTP from rewards alone	Observed at small scale, 3.6–7.1× slower in steps (Section 6)
RL from a verifier learns skills without demonstrations	Observed at small scale; did not exceed automation plus search (Section 7)
One policy is near-optimal in all computable interactive environments	False against informed benchmarks (Proposition 5); weaker notions partly true [35, 37]
RL-native training beats the two-stage pipeline at scale	Open; this paper proposes, it does not show

9 Where the Slogan Stops

"All you need" is a claim about sufficiency in principle. It is important to say exactly where it holds.

Proposition 5 (No universal policy against informed optimal policies)**.** *Let E_L, E_R be two computable environments that look identical before the first action; in E_L choosing left leads to an absorbing state with reward* 1 *forever and right to one with reward* 0*; E_R is the mirror image. A policy choosing left with probability θ earns average reward θ in E_L and $1-\theta$ in E_R, while the informed optimal policy earns* 1 *in both; in one of them the regret grows at least as $T/2$.*

The obstruction is informational and holds for incomputable policies too [35, 36]. Prediction environments escape it because a wrong prediction costs reward but never information. Table 5 summarizes what is and is not established.

Further caveats. The universal policy is incomputable; the universality theorems say nothing about which policy a practical algorithm finds. Our experiments use models of 5–30M parameters, one GPU and mostly one seed; effects at frontier scale may differ in either direction. The miniF2F results compare favorably with a strong automation baseline only in cost, not in solve rate. Finally, Corollary 1 cuts both ways: it shows that NTP is RL, and it also shows that, for data rewards, the best RL algorithm is NTP. The RL-native view gains its content from what it adds to NTP, not from replacing it.

10 Conclusion

Next-token prediction and reinforcement learning are not two paradigms. With a strictly proper scoring rule as reward, the first is the special case of the second in which the environment ignores the agent and the expectation over actions can be taken exactly; the familiar cross-entropy loss is the policy gradient of the logarithmic score. Through this identity the RL framework inherits Solomonoff's universality on prediction tasks, and with a fair two-sample reward it does so even in the token-sampling format that language models use. Ordinary Transformers trained from scalar rewards alone do learn to predict, at the cost per step of NTP and with a statistical price that the theory traces to the scoring rule. And the same framework holds the rewards that imitation cannot use: a proof checker alone taught a small model, shown no proofs, to prove competition problems cheaply, with a proper data reward guarding it against collapse.

In this precise sense, reinforcement learning is all you need: one reward-maximization objective can specify the whole training of a language model, with pretraining as its densest reward. Whether models trained that way will one day surpass the two-stage pipeline is an empirical question that this paper raises rather than settles. The program of Section 8 is our proposal for how to find out.

Code and data. Lean sources, training and evaluation scripts, and all result files are available from the author on request and will accompany the preprint.

A Proofs

Theorem 1. Dominance gives $\ln(\mu^{(n)}/\xi^{(n)}) \le \ln(1/c)$ pointwise on the support of $\mu^{(n)}$, hence $\mathrm{KL}(\mu^{(n)}\|\xi^{(n)}) \le \ln(1/c)$. The chain rule gives $\mathrm{KL}(\mu^{(n)}\|\xi^{(n)}) = \sum_{t\le n} \mathbb{E}_\mu \mathrm{KL}(\mu_t\|\xi_t)$. For $p, q \in \Delta(\mathcal{X})$, $\|p-q\|_2^2 \le \frac{1}{2}\|p-q\|_1^2 \le \mathrm{KL}(p\|q)$ by Pinsker's inequality [38]. The partial sums of $\|\xi_t - \mu_t\|^2$ are non-decreasing with bounded expectations, so the infinite sum is finite almost surely and its terms tend to zero.

Theorem 3. Let $g(z)$ be the gradient of Ω at interior z along T_0. Since interior p maximizes the differentiable J_p, its directional derivatives vanish: $\langle Rp + g(p), v\rangle = 0$ for $v \in T_0$, i.e. $\langle g(z), v\rangle = -v^\top Rz$ for all interior z. For interior p, q and $d = q - p$, $\varphi(s) = \Omega(p + sd)$ has $\varphi'(s) = -d^\top R(p + sd)$, so $\Omega(q) - \Omega(p) = -d^\top Rp - \frac{1}{2}d^\top Rd$. Since $U_\rho(q;p) - U_\rho(p;p) = d^\top Rp$, adding gives $J_p(q) - J_p(p) = -\frac{1}{2}d^\top Rd$; continuity extends this to $q \in \Delta(\mathcal{X})$. For $\Omega = \beta H$ the second derivative along $e_a - e_b$ is $-\beta(1/q(a) + 1/q(b))$, which is not constant, so entropy (and KL, which differs by a linear term) is excluded; and a quadratic divergence is never $\mathrm{KL}(p\|q)$.

Proposition 3. $\mathbb{E}\sum_{k,l} \mathbf{1}\{A_k = A_l\} = K + K(K-1)\|q\|^2$ gives the expression for U. Completing the square, $U = -c\|q - p/c\|^2 + \text{const}$, so the maximizer is the projection, which has the thresholding form with $\tau > 0$ because $\sum_a p(a)/c = 1/c > 1$. Subtracting $\tau > 0$ from $p(a)/c > p(b)/c > \tau$ strictly increases their ratio, and coordinates with $p(a)/c \le \tau$ are clipped to zero.

Proposition 4. $U_\rho(q;p) = \sum_{a_{1:K},x} q(a_1)\cdots q(a_K)\, p(x)\, \rho(a_{1:K}, x)$ is a polynomial, bounded on the compact set $\Delta(\mathcal{X}) \times \Delta(\mathcal{X})$, whereas $\mathrm{KL}(p\|q) \to \infty$ as $q(a) \to 0$ with $p(a) > 0$.

B Experimental Details

Models and optimization. Decoder-only Transformers with learned positional embeddings and pre-layer normalization. Character-level runs: AdamW, learning rate 10^{-3} with 100 warm-up steps and cosine decay, batch 64×128 (Lean) or 64×64 (synthetic), dropout 0.1 on text. BPE runs: width 512, 8 layers, 8 heads, context 512, tied embeddings, AdamW ($\beta = (0.9, 0.95)$, weight decay 0.1), learning rate 6×10^{-4} with 500 warm-up steps and cosine decay to 10%, batch 32×512, bfloat16 autocast. Sampled-token rewards use per-sample credit $r_k = 2\mathbf{1}\{A_k = x\} - w\,\#\{l \ne k : A_l = A_k\}$ with $w = 2/(K-1)$ (fair) or $2/K$ (biased) and a batch-mean baseline.

Verifier RL. Toy propositions: core Lean without Mathlib, one file per candidate, 8 goals $\times$ 8 samples per step, 200 steps, learning rate 10^{-4}. miniF2F one-line: 8 problems $\times$ 16 or 32 samples per step, 60 steps, learning rate 2×10^{-5}, anchor on 8 corpus windows per step; diversity reward shares a unit reward among identical correct proofs in a group and adds a one-time novelty bonus of 0.5. Multi-line: best-first search with 8 policy samples per expansion, at most 6 expansions and depth 6; steps that leave the goal unchanged or revisit a state are pruned; expert iteration uses two rounds of search on `valid` with at most 4 expansions, three epochs over the

accepted steps, anchor $\lambda = 0.5$. Lean checks batch 16 candidates per file under `import Mathlib` with `maxHeartbeats 400000`; candidates containing comments, `sorry`, `admit`, `native_decide`, search commands or declaration keywords are rejected before checking, and a nine-case regression test guards error attribution.

Hardware and software. One NVIDIA RTX PRO 4000 Blackwell laptop GPU (16 GB), 20 CPU cores, PyTorch 2.13, Lean 4.34.1 with Mathlib v4.34.1 for verification and Lean 4.35.0 for core-Lean checks.

References

[1] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. Technical report, OpenAI, 2018.

[2] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[3] Long Ouyang, Jeff Wu, Xu Jiang, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, pages 27730–27744, 2022.

[4] Zhihong Shao, Peiyi Wang, Qihao Zhu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv:2402.03300, 2024.

[5] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv:2501.12948, 2025.

[6] David Silver, Satinder Singh, Doina Precup, and Richard S. Sutton. Reward is enough. *Artificial Intelligence*, 299:103535, 2021.

[7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[8] Ray J. Solomonoff. A formal theory of inductive inference. Part I and Part II. *Information and Control*, 7(1–2):1–22, 224–254, 1964.

[9] Ray J. Solomonoff. Complexity-based induction systems: Comparisons and convergence theorems. *IEEE Transactions on Information Theory*, 24(4):422–432, 1978.

[10] Marcus Hutter. Convergence and error bounds for universal prediction of nonbinary sequences. In *Proc. the 12th European Conference on Machine Learning (ECML)*, volume 2167 of *Lecture Notes in Artificial Intelligence*, pages 239–250. Springer, 2001.

[11] Marcus Hutter. Optimality of universal Bayesian sequence prediction for general loss and alphabet. *Journal of Machine Learning Research*, 4:971–1000, 2003.

[12] Marcus Hutter. *Universal Artificial Intelligence: Sequential Decisions Based on Algorithmic Probability*. Springer, Berlin, 2005.

[13] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.

[14] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2nd edition, 2018.

[15] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.

[16] Leonard J. Savage. Elicitation of personal probabilities and expectations. *Journal of the American Statistical Association*, 66(336):783–801, 1971.

[17] Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.

[18] Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.

[19] Ming Li and Paul Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Springer, Cham, 4th edition, 2019.

[20] Jan Leike and Marcus Hutter. On the computability of Solomonoff induction and AIXI. *Theoretical Computer Science*, 716:28–49, 2018.

[21] Christopher A. T. Ferro. Fair scores for ensemble forecasts. *Quarterly Journal of the Royal Meteorological Society*, 140(683):1917–1923, 2014.

[22] Tomasz Korbak, Ethan Perez, and Christopher L. Buckley. RL with KL penalties is better viewed as Bayesian inference. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 1083–1091, 2022.

[23] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.

[24] Anthony GX-Chen, Jatin Prakash, Jeff Guo, Rob Fergus, and Rajesh Ranganath. KL-regularized reinforcement learning is designed to mode collapse. arXiv:2510.20817, 2025.

[25] Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025.

[26] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Proc. the 28th International Conference on Automated Deduction (CADE)*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[27] The mathlib Community. The Lean mathematical library. In *Proc. the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pages 367–381, 2020.

[28] Alexey Milovanov. Formalized algorithmic information theory in Lean 4. Software, `https://reservoir.lean-lang.org/@AlexeyMilovanov/kolmogorov_complexity`, accessed Oct. 2026, 2026.

[29] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. MiniF2F: a cross-system benchmark for formal Olympiad-level mathematics. In *International Conference on Learning Representations*, 2022.

[30] Huajian Xin, Z. Z. Ren, Junxiao Song, Zhihong Shao, Wanjia Zhao, Haocheng Wang, Bo Liu, Liyue Zhang, Xuan Lu, Qiushi Du, Wenjun Gao, Qihao Zhu, Dejian Yang, Zhibin Gou, Z. F. Wu, Fuli Luo, and Chong Ruan. DeepSeek-Prover-V1.5: Harnessing proof assistant feedback for reinforcement learning and Monte-Carlo tree search. arXiv:2408.08152, 2024.

[31] Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[32] Thomas Anthony, Zheng Tian, and David Barber. Thinking fast and slow with deep learning and tree search. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[33] Stanislas Polu, Jesse Michael Han, Kunhao Zheng, Mantas Baksys, Igor Babuschkin, and Ilya Sutskever. Formal mathematics statement curriculum learning. In *International Conference on Learning Representations*, 2023.

[34] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2023.

[35] Tor Lattimore and Marcus Hutter. Asymptotically optimal agents. In *Proc. the 22nd International Conference on Algorithmic Learning Theory (ALT)*, volume 6925 of *Lecture Notes in Computer Science*, pages 368–382. Springer, 2011.

[36] Jan Leike and Marcus Hutter. Bad universal priors and notions of optimality. In *Proc. the 28th Conference on Learning Theory (COLT)*, volume 40 of *Proceedings of Machine Learning Research*, pages 1244–1259, 2015.

[37] Jan Leike, Tor Lattimore, Laurent Orseau, and Marcus Hutter. Thompson sampling is asymptotically optimal in general environments. In *Proc. the 32nd Conference on Uncertainty in Artificial Intelligence (UAI)*, 2016.

[38] Imre Csiszár. Information-type measures of difference of probability distributions and indirect observations. *Studia Scientiarum Mathematicarum Hungarica*, 2:299–318, 1967.